Comprobador de paquetes HL7

1. Motivaciones

En la actualidad, los sistemas de telemonitorización han obtenido mayor peso en el ámbito sanitario debido a los nuevos retos médicos, como el aumento de la cronificación de enfermedades, así como por los avances tecnológicos que permiten crear dispositivos de menor coste, tamaño y consumo energético. En consecuencia, se está buscando una correcta integración de sensores, sistemas de comunicación inalámbrica y protocolos interoperables con el objetivo de mejorar el seguimiento continuo y remoto de parámetros fisiológicos puesto que mejoraría la prevención, el diagnóstico y el seguimiento de las patologías.

En este contexto, el estándar HL7 v2.6 es esencial para lograr la interoperabilidad entre dispositivos médicos y sistemas de información hospitalarios dado que ofrece el consenso de un marco completo para el intercambio de datos de salud. Por tanto, este proyecto pretende ser de utilidad para comprender mejor la estructura de este estándar mediante la implementación de un flujo completo de comunicación sensor-gateway-PC donde se generen, transmitan y validen mensajes HL7 contextualizados en una toma a tiempo real de datos de señales biomédicas.

2. Objetivos

El objetivo principal es desarrollar un comprobador sintáctico de paquetes HL7 capaz de solicitar, recibir, analizar y representar información fisiológica proveniente de un nodo sensor a través de un gateway. Para ello, se establecen los siguientes objetivos específicos:

- Implementar flujo de comunicación bidireccional entre Bitalino-Sensor-Gateway-PC
- Desarrollar una interfaz gráfica en Python que permita seleccionar el tipo de mensaje y error.
- Implementar dos botones en la interfaz gráfica para trasladar al sensor mensajes HL7 de tipo Z que marquen la petición de inicio de comunicación y, en el caso de estar empleando mensajes ORU_R01, también, la petición de finalización.
- Procesar los mensajes Z en el nodo sensor y generar los mensajes HL7 pedidos, que serán de tipo ORU_R01, ADT_A05 y OMD_O03. El mensaje ORU_R01 se conformará con los parámetros biomédicos obtenidos desde el Bitalino.
- En la interfaz, visualizar el texto del mensaje recibido y, si el mensaje es ORU_R01, representar los parámetros medidos en gráficas en tiempo real donde se puedan establecer umbrales que generen alertas visuales.
- Revisar en la interfaz los distintos campos de los mensajes HL7 enviados por el sensor y detectar errores. A partir de la validación, generar mensajes HL7 de tipo ACK con la codificación del error detectado, si procede.
- Procesar mensajes ACK en el sensor y decidir detener el envío de mensajes si el ACK indica un error detectado o si el ACK corresponde a un mensaje de tipo no ORU_R01, independientemente de si se haya detectado error.

3. Materiales

Para la realización de este proyecto se han utilizados los siguientes materiales:

- Bitalino junto con los sensores para adquirir datos biológicos: temperatura, pulsioxímetro y sensor de aceleración.

- ESP32 - Nodo Sensor: encargado tanto de la recepción de datos desde el Bitalino como de la generación de los distintos mensajes HL7.

- ESP32 - Nodo Gateway: actúa como un mero intermediario que recibe los mensajes del nodo Sensor y los reenvía al PC, y viceversa, sin realizar un procesamiento adicional sobre la información.

A nivel de software, se ha utilizado Python 3.11, para el desarrollo de la interfaz gráfica donde se han empleado librerías como PyQt5, PySerial, Matplotlib y Numpy. Paralelamente, se ha usado Arduino IDE, para la programación de ambos ESP32 donde se implementa la conexión mediante Bluetooth y las funciones correspondientes al Sensor y al Gateway.

4. Resultados y discusión

El sistema implementado cumple con los objetivos definidos anteriormente; el nodo Sensor genera los distintos mensajes en HL7, el Gateway actúa como un puente de comunicación entre dispositivos y el PC gestiona tanto el envío de mensajes Z y ACK, como la recepción y la validación de los mensajes procedentes del nodo Sensor. Asimismo, implementa toda la visualización, incluyendo la monitorización en tiempo real y la gestión de errores, junto con la interacción con el usuario.

El flujo de comunicación generado se ha reproducido y validado en diferentes pruebas, verificando, por un lado, que ante la recepción de un mensaje no ORU o ante un mensaje con error, se produce la parada automática de la comunicación y, por otro lado, si el mensaje recibido se trata de un mensaje ORU con error, se transmite y se gráfica de forma continua hasta la llegada de un mensaje Z final.

A nivel de interfaz gráfica, en la figura 1, se puede comprobar la organización de la ventana principal en diversas secciones. En primer lugar, se incluye la existencia de un panel de control general con dos botones encargados de producir los respectivos mensajes Z de inicio y de fin. Asimismo, existe la posibilidad de configuración de este mensaje inicial mediante desplegables para la selección del tipo de mensaje que se quiere enviar, la presencia y el tipo de errores junto al número de sensores que se desea monitorizar, con la posibilidad de introducir umbrales mínimos y máximos cuya superación activa un aviso para alertar de un posible riesgo para el paciente. Cabe mencionar que la interfaz muestra tanto una visualización previa del mensaje a enviar, permitiendo verificar su estructura antes de transmitirlo, como el flujo completo de comunicación en tiempo real en el Gateway Log.

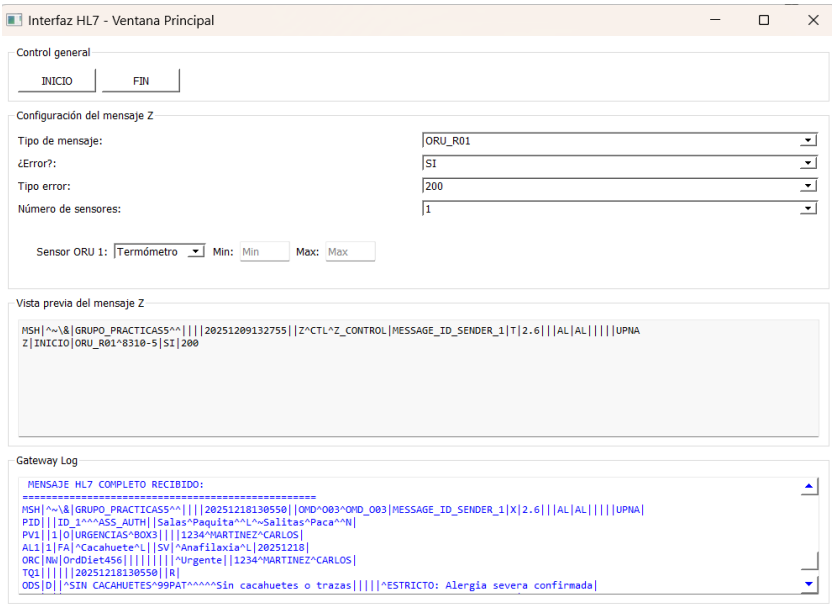


Figura 1: Visualización de la ventana principal de la Interfaz Gráfica

Respecto al flujo de comunicación de mensajes ORU, el programa está diseñado para que se abran tantas ventanas emergentes como sensores se hayan seleccionado anteriormente, independientemente de si el mensaje contiene errores o no, ya que, el envío de errores ORU se realiza a nivel global. De forma general, en cada ventana emergente aparece información general del tipo de sensor, el mensaje recibido y dos secciones referidas al procesamiento de errores, donde no solo aparece el tipo de mensaje solicitado desde la interfaz, sino que también la detección de ausencia o presencia de error, respectivamente en un cuadro verde o rojo.

En el caso de seleccionar un mensaje ORU sin error, la interfaz está configurada para ser capaz de procesar hasta cuatro tipos de sensores diferentes: acelerómetro, termómetro, pulsioxímetro y tensiómetro. La graficación para la monitorización en tiempo real se ha realizado de tal forma que es capaz de ajustarse al rango de entrada de los valores de los sensores y la actualización de los distintos valores adquiridos se realiza cada cinco segundos. Asimismo, como se puede observar en la siguiente imagen, cuando el valor recibido de un sensor queda por debajo del umbral mínimo o supera el umbral máximo, el sistema genera un aviso temporal para informar al sanitario. Sin embargo, en el caso de un mensaje ORU con error es importante mencionar que aunque la gráfica se muestra, esta aparece siempre vacía.

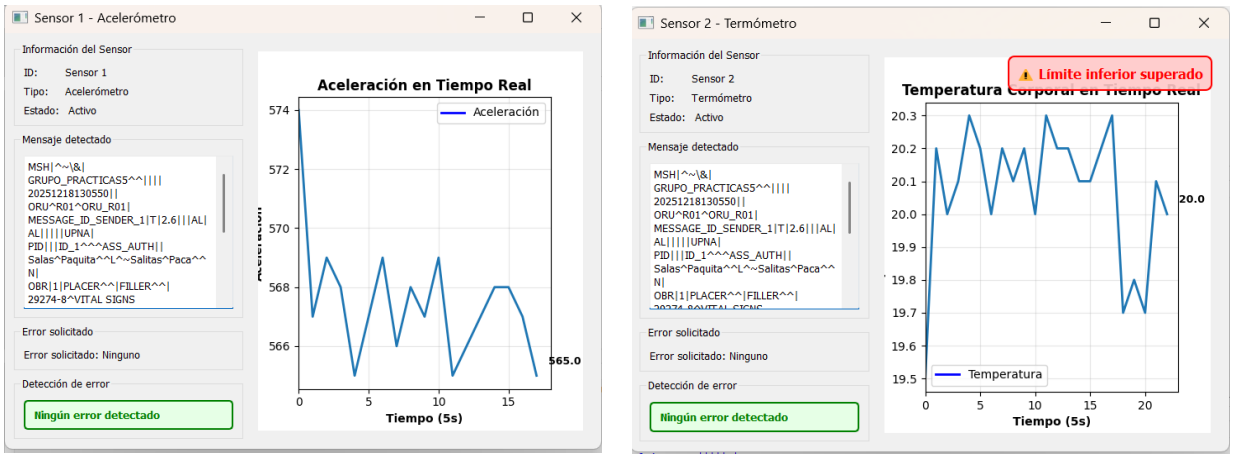


Figura 2: Visualización de las ventanas propias de los mensajes ORU sin errores donde se muestra la obtención de datos en tiempo real, el mensaje recibido, la ausencia de errores y los avisos pertinentes.

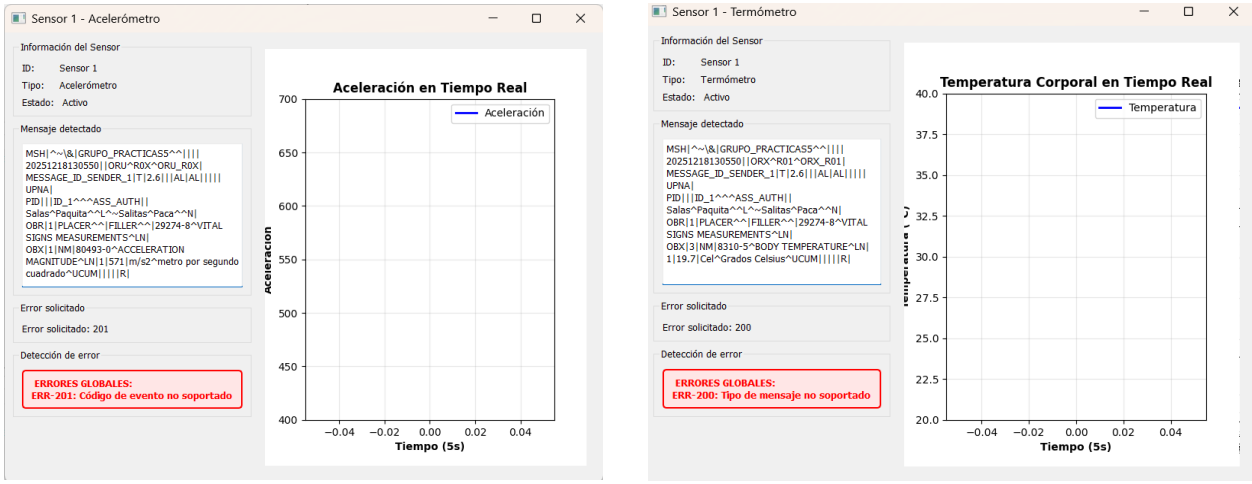


Figura 3: Visualización de las ventanas propias de los mensajes ORU con error donde se observa el mensaje recibido, la detección de error y el tipo de error.

En cuanto a los mensajes no ORU, como ADT_A05 y OMD_O03, de forma muy análoga al procesamiento de mensajes ORU, el programa abre ventanas emergentes que muestran el contenido del mensaje recibido. No obstante, en este caso, contienen únicamente el mensaje recibido y la detección de presencia o ausencia de errores, siguiendo el mismo código de colores que el utilizado anteriormente.

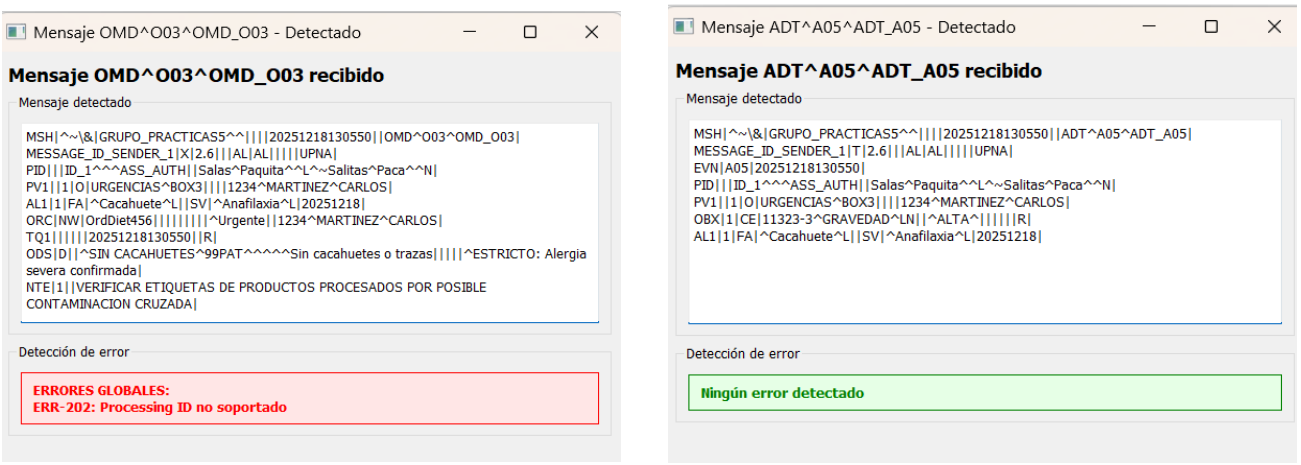


Figura 4: Visualización de los mensajes no ORU, la ventana de la izquierda se trata de un mensaje ADT-A05 sin error mientras que el mensaje de la derecha corresponde a un mensaje OMD-O03 con error.

Asimismo, es importante destacar que la detección de errores es independiente del mensaje solicitado desde la interfaz. De tal forma que si el mensaje solicitado no presenta errores, pero el sensor lo envía con errores, la interfaz detectará el tipo de error, mandará el correspondiente ACK y finalizará la comunicación, garantizando la seguridad y coherencia del sistema. Teniendo esto en cuenta, la interfaz permite identificar de forma precisa las anomalías presentes en el mensaje HL7. Entre los errores gestionados se encuentran el error 101, campo requerido ausente, se genera cuando falta un campo obligatorio; el error 200, tipo de mensaje no soportado, se activa cuando el campo MSH-9 contiene un tipo de mensaje distinto a los permitidos por la interfaz ; el error 201, código de evento no soportado, aparece cuando la segunda parte del identificador del mensaje incluye un código no válido; el error 202, ID no soportado, se detecta cuando el campo MSH-11 presenta un valor no admitido, y el error 203, versión de HL7 no soportada, se activa al recibir un mensaje cuya versión del estándar no coindice con la admitida.

Recapitulando lo mencionado anteriormente, el desarrollo del comprobador de paquetes HL7 ha permitido evaluar la complejidad presente en los sistemas de telemonitorización y, especialmente, la importancia de disponer de mecanismos de validación fiables en entornos interoperables. Los resultados obtenidos demuestran que el sistema desarrollado es capaz de reproducir con fidelidad el flujo de comunicación definido en el proyecto y gestionar de manera adecuada tanto los mensajes correctos como aquellos que contienen errores.

El enfoque modular, basado en la separación entre Sensor, Gateway y PC, facilita la trazabilidad de cada paquete y la verificación independiente de su estructura, ofreciendo un flujo de comunicación estable y predecible. Para ello, durante la comunicación entre el PC y el Gateway se emplea el protocolo MLLP (Minimal Lower Layer Protocol), que permite encapsular los mensajes HL7 mediante delimitadores específicos: VT (0x0B) al inicio y FS+CR (0x1C 0x0D) al final. Además, el Gateway y la interfaz procesan el flujo de datos byte a byte, acumulando el contenido entre estos delimitadores antes de validar y representar el mensaje.

No obstante, durante las pruebas se identificaron ciertas limitaciones que conviene destacar: la detección de errores depende estrictamente de la integridad sintáctica del mensaje recibido, por lo que errores semánticos o valores clínicamente incoherentes no son evaluados por el sistema y serían necesarios en un escenario real para evitar interpretaciones erróneas de datos biomédicos.

Por otro lado, aunque el uso de mensajes Z es adecuado para la práctica, podría comprometer la interoperabilidad con plataformas hospitalarias reales, lo que abre la puerta a futuras mejoras orientadas a la conformidad estricta con el estándar. Además, las gráficas de monitorización están limitadas por la frecuencia de actualización y por el rango dinámico de los sensores utilizados, por lo que su escalabilidad es limitada si se pretende integrar un mayor número de sensores o flujos simultáneos.

Aun así, en conjunto, el comportamiento global del sistema es robusto y cumple de manera satisfactoria los requisitos establecidos, constituyendo una base sólida sobre la que integrar nuevos tipos de mensajes, sensores o validaciones clínicas más avanzadas.

5. Conclusiones

En el desarrollo del trabajo se ha adquirido un dominio práctico de los sistemas de telemonitorización, así como del estándar HL7 v2.6. Para lo cual, se ha abordado todas las partes relativas a una implementación de un sistema de telemonitorización: comunicación inalámbrica, utilización precisa del estándar HL7, verificación sintáctica, gestión de errores, graficado de parámetros biomédicos en tiempo real mediante una interfaz gráfica amigable para el usuario.

Asimismo, previo a la fase de desarrollo, ha sido necesario comprender el correcto manejo de la documentación relativa al estándar y a los códigos LOINC y UCUM. De esta manera, se ha observado la importancia tanto de un consenso en el intercambio de información clínica como de una documentación precisa y específica que recoja dicho consenso. Sin este paso inicial, no se podría garantizar la interoperabilidad del comprobador de mensajes implementado en este trabajo impidiendo su integración fiable en un ecosistema clínico real donde se necesita asegurar coherencia, trazabilidad y validez en los datos biomédicos intercambiados.

Por otro lado, con el proceso de validación e identificación de los distintos tipos de errores sintácticos en mensajes HL7 tratados en el trabajo se obtuvo una mayor comprensión

del funcionamiento interno del estándar y de la relevancia de la detección temprana de incompatibilidades o incongruencias en flujos de intercambio de datos clínicos. Un sistema de identificación autónoma de errores en la estructura del mensaje añade una capa adicional de seguridad de gran importancia en la práctica clínica.

Por último, en el trabajo se han encontrado ciertos desafíos reales asociados al manejo de los sistemas de telemonitorización clínicos. Aunque se ha logrado desarrollar un sistema funcional que cumple los objetivos propuesto, también, se han identificado limitaciones inherentes a las tecnologías empleadas como la fragilidad en el establecimiento inicial de la comunicación inalámbrica o el reto de minimizar el tiempo de envío y procesado para procurar un seguimiento remoto a un tiempo lo más ajustado al real posible.

6. Bibliografía

- HL7 International. (2007). *HL7 Version 2.6 Standard*. Health Level Seven International.
- Regenstrief Institute. (2014, February 6). *Table of Example UCUM codes for Electronic Messaging (Version 1.2)*. The Unified Code for Units of Measure (UCUM).
- Regenstrief Institute. (2024). *LOINC® (Logical Observation Identifiers Names and Codes) Database*. Regenstrief Institute.

ANEXO
Mensajes HL7 creados en el sensor

Cabe señalar que los mensajes tipo ORU_R01 expuestos se han diseñado con 4 segmentos OBX diferentes para cada sensor disponible. No obstante, el mensaje final creado tendrá tantos segmentos OBX como sensores se hayan seleccionado desde la interfaz, con un máximo de 4 sensores. En estos segmentos, el valor *parametromedido* corresponde al dato numérico recogido del sensor.

a) Sin errores

ORU_R01 - Unsolicited transmission of an observation message:

MSH | ^~\& | GRUPO_PRACTICAS5^^ | | | | 20251218130550 | | ORU^R01^ORU_R01 | MESSAGE_ID_SENDER_1 | T | 2.6 | | | AL | AL | | | | | UPNA
*PID | | | ID_1^^^ASS_AUTH^^^^^^ | | Salas^Paquita^^^L^~Salitas^Paca^^^N^*
OBR | 1 | PLACER^^^^ | FILLER^^^^ | 29274-8^VITAL SIGNS MEASUREMENTS^LN
OBX | 1 | NM | 80493-0^ACCELERATION MAGNITUDE^LN | 1 | parametromedido | m/s2^metro por segundo cuadrado^UCUM|||||R
OBX | 2 | NM | 20564-1^OXYGEN SATURATION IN BLOOD^LN | 1 | parametromedido | %{saturation}^porcentaje^UCUM | | | | | R
OBX | 3 | NM | 8310-5^BODY TEMPERATURE^LN | 1 | parametromedido | Cel^Grados Celsius^UCUM | | | | | R
OBX | 4 | NM | 55284-4^BLOOD PRESSURE^LN | 1 | parametromedido | mm[Hg]^milímetro de mercurio^UCUM | | | | | R

OMD_O03 - Dietary order:

MSH | ^~\& | GRUPO_PRACTICAS5^^ | | | | 20251218130550 | | OMD^O03^OMD_O03 | MESSAGE_ID_SENDER_2 | T | 2.6 | | | AL | AL | | | | | UPNA
*PID | | | ID_1^^^ASS_AUTH^^^^^^ | | Salas^Paquita^^^L^~Salitas^Paca^^^N^*
PV1 | 1 | O | URGENCIAS^BOX3 | | | | 1234^MARTINEZ^CARLOS
AL1 | 1 | FA | ^Cacahuete^L | | SV | ^Anafilaxia^L | 20251218
ORC | NW | OrdDiet456 | | | | | | | | ^Urgente | | 1234^MARTINEZ^CARLOS
TQ1 | | | | | | 20251218130550| | R
ODS | D | | ^SIN CACAHUETES^99PAT^^^^^Sin cacahuetes o trazas | | | | | ^ESTRICTO: Alergia severa confirmada
NTE | 1 | | VERIFICAR ETIQUETAS DE PRODUCTOS PROCESADOS POR POSIBLE CONTAMINACION CRUZADA

ADT_A05 - Pre-Admit a Patient:

MSH | ^~\& | GRUPO_PRACTICAS5^^ | | | | 20251218130550 | | ADT^A05^ADT_A05 | MESSAGE_ID_SENDER_3 | T | 2.6 | | | AL | AL | | | | | UPNA
EVN | A05 | 20251218130550
*PID | | | ID_1^^^ASS_AUTH^^^^^^ | | Salas^Paquita^^^L^~Salitas^Paca^^^N^*
PV1 | 1 | O | URGENCIAS^BOX3 | | | | 1234^MARTINEZ^CARLOS
OBX | 1 | CE | 11323-3^GRAVEDAD^LN | 1 | ^ALTA^| | | | | | R
AL1 | 1 | FA | ^Cacahuete^L | | SV | ^Anafilaxia^L | 20140402

b) Con errores

Error 200: tipo de mensaje no soportado. Cambia solo la creación del segmento MSH, el valor en negrita tomará el valor de ORX, ADX u OMX, según el tipo de mensaje. El resto de segmentos quedarían igual. A modo de ejemplo, se muestra solo el segmento MSH del mensaje ORU_R01.

*MSH | ^~\& | GRUPO_PRACTICAS5 | | | | 20251218130550 | | **ORX**^R01^**ORX**_R01 | MESSAGE_ID_SENDER_1 | T | 2.6 | | | AL | AL | | | | | UPNA*

Error 201: código de evento no soportado. Cambia solo la creación del segmento MSH, el valor en negrita tomará el valor de R0X, A0X u O0X, según el tipo de mensaje. El resto de segmentos quedarían igual. A modo de ejemplo, se muestra solo el segmento MSH del mensaje ORU_R01.

*MSH | ^~\& | GRUPO_PRACTICAS5 | | | | 20251218130550 | | ORU^**R0X**^ORU_**R0X** | MESSAGE_ID_SENDER_1 | T | 2.6 | | | AL | AL | | | | | UPNA*

Error 202: Processing ID no soportado (valores válidos son T, D o P). Cambia solo la creación del segmento MSH, el resto del mensaje sería igual en los tres casos.

MSH | ^~\& | *GRUPO_PRACTICAS5^^* | | | | *20251218130550* | | *ADT^A05^ADT_A05* | *MESSAGE_ID_SENDER_3* | ***X*** | *2.6* | | | *AL* | *AL* | | | | | *UPNA*

Error 203: versión no soportada. Cambia solo la creación del segmento MSH, el resto del mensaje sería igual en los tres casos.

MSH | ^~\& | *GRUPO_PRACTICAS5^^* | | | | *20251218130550* | | *ADT^A05^ADT_A05* | *MESSAGE_ID_SENDER_3* | *T* | ***3.0*** | | | *AL* | *AL* | | | | | *UPNA*

Error 101: falta un campo requerido. Como en los tres tipos de mensajes tienen segmento PID, se optó por eliminar el campo PID-3.

PID | | | | *Salas^Paquita^^^L^~Salitas^Paca^^^N^*

No obstante, la interfaz comprueba todos los campos requeridos de los segmentos posibles que puede recibir:

Tipo segmento	**Campos requeridos**
MSH	MSH-1, MSH-2, MSH-3, MSH-7, MSH-9, MSH-10, MSH-11, MSH-12, MSH-15, MSH-16, MSH-21
PID	PID-3, PID-5
OBR	OBR-1, OBR-2, OBR-3, OBR-4
OBX	OBX-1, OBX-3, OBX-4, OBX-11
PV1	PV1-2
AL1	AL1-1, AL-3
ORC	ORC-1
TQ1	-
ODS	ODS-1, ODS-3
NTE	NTE-1

Mensajes HL7 creados en el PC

a) Z control

MSH | ^~\& | *GRUPO_PRACTICAS5^^* | | | | *20140402130550* | | *Z^CTL^Z_CONTROL*| *MESSAGE_ID_SENDER_1* | *T* | *2.6* | | | *AL* | *AL* | | | | | *UPNA*
Z | ***Z-1*** | ***Z-2*** | ***Z-3*** | ***Z-4***

Donde los campos en negrita pueden tomar los siguientes valores:

- Z-1: INICIO o FINAL
- Z-2: ORU_R01, ADT_A05 o OMD_O03. Si es ORU_RO1, se le añadirán subcampos con los códigos LOINC de los sensores que se vayan a solicitar (ej. ORU_R01^20564-1^8310-5). Este campo no se crea si el valor de Z-1 es FINAL.
- Z-3: SI o NO. Este campo no se crea si el valor de Z-1 es FINAL
- Z-4: 200, 201, 202, 203 o 101. Este campo no se crea si el valor de Z-1 es FINAL o si el valor de Z-3 es NO

b) ACK

Sin error:

MSH | ^~\& | *GRUPO_PRACTICAS5^^* | | ***MSH-3_INCOMING (sender)*** | | *20251218130550*| | *ACK^**A**^ACK* | *ACK_sender_id_message* | *T* | *2.6* | | | | | | | | *UPNA*
MSA | *AA* | ***MSH-10_INCOMING (sender)***

Con error:

MSH | ^~\& | *GRUPO_PRACTICAS5^^* | | ***MSH-3_INCOMING (sender)*** | | *20251218130550* | | | *ACK^**A**^ACK* | *ACK_sender_id_message* | *T* | *2.6* | | | | | | | | *UPNA*
MSA | *AR* | ***MSH-10_INCOMING (sender)***
ERR | | | ***B*** | *E*

Para generar los ACKs, los campos en negrita se obtienen de procesar el segmento MSH del mensaje recibido que genera el ACK. Por otro lado, los campos en verde pueden tomar los siguientes valores:

- A: R01 (si es ORU_R01), A05 (si es ADT_A05) u O03 (si es OMD_O03)
- B: 200, 201, 202, 203 o 101